Run the sjSDM Cross-Validation Preparation Sequence

Description

Builds selected cross-validation preparation targets in isolated unit stores without requesting tuning fits or final models.

Usage

```
run_sjsdm_cv_preparation_sequence(
  unit_pipeline = NULL,
  preparation_target_names = NULL,
  unit_store_suffixes = NULL,
  prebuild_interpolation = FALSE,
  preparation_resource_profile = "shared",
  interpolation_workers_override = NULL,
  run_pipeline_function = run_pipeline,
  target_store = load_active_config_value("target_store"),
  resolve_store_function = resolve_pipeline_store_path,
  load_metadata_function = load_targets_store_metadata,
  classify_error_function = classify_sjsdm_unit_pipeline_error,
  diagnose_endpoints_function = diagnose_sjsdm_cv_preparation_endpoints,
  read_target_function = targets::tar_read_raw,
  invalidate_target_function = targets::tar_invalidate,
  validate_endpoints = base::identical(run_pipeline_function, run_pipeline),
  cache_recovery_attempts = 8L
)
```

Arguments

`unit_pipeline`	Character scalar path to the production unit pipeline.
`preparation_target_names`	Non-empty character vector of prepared-fold target names.

`unit_store_suffixes`	Optional character vector of spatial-unit store suffixes. 'NULL' runs the non-nested store used by a temporal profile.
`prebuild_interpolation`	Logical forwarded to [run_pipeline()] for each unit execution.
`preparation_resource_profile`	Runtime resource profile forwarded to [run_pipeline()].
`interpolation_workers_override`	Optional explicit positive interpolation worker count.
`run_pipeline_function`	Injectable pipeline runner. Defaults to [run_pipeline()].
`target_store`	Root target-store path used to inspect unit-level target errors.
`resolve_store_function`	Injectable unit-store path resolver.
`load_metadata_function`, `classify_error_function`	Injectable target-metadata loader and unit-error classifier.
`diagnose_endpoints_function`	Injectable prepared-endpoint diagnostic.
`read_target_function`	Injectable raw target reader used by endpoint validation.
`invalidate_target_function`	Injectable target invalidator used for legacy-cache recovery.
`validate_endpoints`	Logical. If 'TRUE', require every requested endpoint to have successful, hashed metadata and to be readable. Defaults to 'TRUE' for the production pipeline runner and 'FALSE' for injected test runners.
`cache_recovery_attempts`	Non-negative integer maximum number of targeted legacy-cache retries.

Details

This function is used only when a main-analysis runner is explicitly placed in CV-fold preparation mode. A target-level failure in one spatial unit or temporal store is recorded so all remaining independent work can be attempted. Expected infeasibility is accepted. Unexplained target failures and per-unit runner failures without fresh target metadata are reported after all spatial units have been attempted. This guarantees maximum unit coverage while keeping unexpected outcomes visible at component exit. Legacy-cache recovery follows all

affected targets and dependencies visible in metadata, but stops for registered data limitations or when an invalidation makes no progress. Final unexplained-error output is grouped once per unit.

Value

Invisibly, a unit-status tibble. Prepared targets remain cached in production stores.